Algorithms 2

Mingxun Zhou (mingxunz@ust.hk)

COMP2711 Discrete Mathematical Tools

Learning objectives

After this lecture, you should be able to:

1. **Sorting.** Explain selection sort and merge sort, and analyze their running times.
2. **Searching.** Use linear search and binary search; identify when binary search applies.
3. **Preprocessing.** Compare the total cost of answering many queries.
4. **Space cost.** Measure extra memory and explain the time–space tradeoff.

1 Sorting

The sorting problem asks us to rearrange $A[1..n]$ into nondecreasing order while preserving all its values, including repetitions. We count constant-time comparisons and operations on machine-sized values [1, Sections 3.1–3.3].

1.1 Selection sort

Selection sort repeatedly finds the smallest remaining value and puts it in the next position. For example,

$$[4, 2, 5, 1, 3] \to [1 \mid 2, 5, 4, 3] \to [1, 2 \mid 5, 4, 3] \to [1, 2, 3 \mid 4, 5].$$

The bar separates the fixed prefix from the unsorted suffix.

SelectionSort(A)
for $i = 1, \ldots, n-1$:
  $p \leftarrow i$
  for $j = i+1, \ldots, n$:
    if $A[j] < A[p]$: $p \leftarrow j$
  if $p \neq i$: swap $A[i], A[p]$

Why it works. Pass i puts the ith smallest value in position i. Earlier positions stay fixed; after $n-1$ passes, the array is sorted.

Pass i uses $n - i$ value comparisons. Therefore, on every input,

$$C(n) = \sum_{i=1}^{n-1} (n - i) = \frac{n(n-1)}{2}.$$

The running time is $\Theta(n^2)$, even for sorted input. There are at most $n - 1$ swaps and $O(1)$ auxiliary space.

1.2 Merge sort

MUST UNDERSTAND Divide and conquer

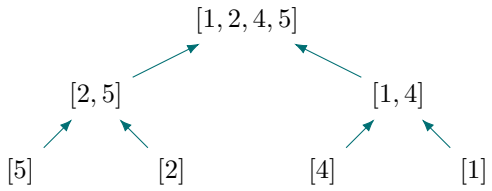
Divide an instance into smaller instances, solve those, and combine their answers. **Merge sort** splits a list into two nearly equal halves, sorts both halves, and merges them. A list with at most one element is already sorted.

MergeSort(A)
if $|A| \leq 1$: **return** A
split A into two nearly equal halves L, R
$L \leftarrow \text{MergeSort}(L)$; $R \leftarrow \text{MergeSort}(R)$
return $\text{Merge}(L, R)$

The calls apply the same algorithm to smaller inputs; this is **recursion**. The sizes decrease until the base case.

Merge two sorted lists. Compare their first unconsumed values, append the smaller to the output, and advance that list's index. When one list is exhausted, append the other list's remainder. For $[2, 5]$ and $[1, 4]$, take 1, then 2, then 4, and append 5.

Why merging works. Each step outputs the smallest remaining value exactly once. For two nonempty lists of total length k, merging takes at most $k - 1$ comparisons and $\Theta(k)$ time.



Reading upward, sorted singleton lists merge into sorted pairs, then into the sorted whole list. This also proves correctness: each parent is sorted and contains exactly the values of its children.

MUST UNDERSTAND Counting work by levels

For $n = 2^h$, merge sort has $h = \log_2 n$ merge levels. At a level with 2^d lists of size $n/2^d$, the total work is

$$2^d \times \Theta(n/2^d) = \Theta(n).$$

$$\text{Total time} = \Theta(n) \times \log_2 n = \Theta(n \log n).$$

For general n, there are between $\lfloor \log_2 n \rfloor$ full merge levels and $\lceil \log_2 n \rceil$ levels in total. The same bound holds. Extra storage is $O(n)$ [1, Section 5.4].

2 Searching

Given $A[1..n]$ and target x, return a matching index, or -1 if x is absent. Duplicates are allowed. The array supports constant-time indexed access.

2.1 Linear search

Check positions $1, 2, \ldots, n$ and return the first match. Every failed test rules out one position; exhausting the array proves absence.

LinearSearch(A, x)
for $i = 1, \ldots, n$:
  if $A[i] = x$: **return** i
return -1

For $[8, 3, 12, 5, 9, 2, 7]$, target 9 is found at position 5 after five tests. Target 6 needs all seven tests and returns -1. The worst-case time is $\Theta(n)$; the best case is $\Theta(1)$ when the first value matches.

2.2 Binary search

MUST UNDERSTAND Binary search and its precondition

Binary search requires a sorted array. Compare the target with the middle value and discard the half that cannot contain a match.

BinarySearch(A, x)
$L \leftarrow 1$; $R \leftarrow n$
while $L \le R$:
  $M \leftarrow \lfloor (L + R)/2 \rfloor$
  if $A[M] = x$: **return** M
  if $A[M] < x$: $L \leftarrow M + 1$
  else: $R \leftarrow M - 1$
return -1

Why it works. If $A[M] < x$, everything at or left of M is too small; the other direction is symmetric. A possible match stays in the shrinking interval. An empty interval means no match. Any matching index may be returned.

For $A = [2, 3, 5, 7, 8, 9, 12]$:

Target	Probed positions	Result
9	$4, 6$ (values $7, 9$)	Return 6.
6	$4, 2, 3$ (values $7, 3, 5$)	Return -1.

Cost. Each failed probe halves the candidates. The worst case uses $\lfloor \log_2 n \rfloor + 1$ probes (attained by a target larger than every value). Thus time is $\Theta(\log n)$ for $n \ge 2$, and extra space is $O(1)$.

3 Preprocessing

MUST UNDERSTAND Preprocessing

Preprocessing is work performed once to make later queries cheaper. Its cost belongs in the total running time.

Suppose an unchanged, initially unsorted array has $n \geq 2$ values, and we answer q membership queries. Compare two strategies:

	Repeated linear search	Sort once, then binary search
Preparation	None	$\Theta(n \log n)$
Worst case per query	$\Theta(n)$	$\Theta(\log n)$
Total worst-case time	$\Theta(qn)$	$\Theta(n \log n + q \log n)$

- $q = 1$: scanning is cheaper asymptotically.
- $q = \lceil \log_2 n \rceil$: both costs are $\Theta(n \log n)$.
- $q = n$: scanning costs $\Theta(n^2)$; sorting once costs $\Theta(n \log n)$.

4 Space cost

MUST UNDERSTAND Auxiliary space

Auxiliary space is the maximum extra memory used at one time, beyond the input. Both iterative search algorithms use $O(1)$ extra space.

4.1 Direct addressing

If all values are integers in $\{1, \ldots, U\}$, initialize a Boolean array $Present[1..U]$ to No, then set $Present[x]$ to Yes for every input value x. Query x by reading that entry; a query outside the range returns No.

For $A = [2, 5, 2, 7]$ and $U = 8$,

x	1	2	3	4	5	6	7	8
$Present[x]$	No	Yes	No	No	Yes	No	Yes	No

Initialization visits U entries; insertion visits n input values. Each query reads one entry. The table needs space for all U possible values, even those absent from the input.

Method	Preparation	Query time	Extra space
Linear scan	None	$O(n)$	$O(1)$
Merge sort + binary search	$O(n \log n)$	$O(\log n)$	$O(n)$
Direct addressing	$O(U + n)$	$O(1)$	$O(U)$

Space includes preparation; the merge-sort row refers to the copying implementation taught here. If $U = O(n \log n)$, direct addressing fits an $O(n \log n)$ preparation budget and gives constant-time queries.

5 Additional sorting and matching examples

5.1 Insertion sort and bubble sort

Insertion sort keeps a sorted prefix. Save the next value, shift larger prefix entries right, and insert the saved value in the gap. For example,

$$[5 \mid 2, 4, 1] \to [2, 5 \mid 4, 1] \to [2, 4, 5 \mid 1] \to [1, 2, 4, 5].$$

Every step preserves the prefix's values and order. On reverse-sorted input, pass i compares against all $i - 1$ earlier values; on sorted input, it needs one comparison.

Bubble sort compares adjacent values and swaps inverted pairs. A pass moves the largest remaining value into its final position. On the same example, the first pass is

$$[5, 2, 4, 1] \to [2, 5, 4, 1] \to [2, 4, 5, 1] \to [2, 4, 1, 5].$$

Fixed passes of lengths $n - 1, n - 2, \ldots, 1$ give quadratic time on every input. Stopping after a pass with no swaps gives a different version with linear best-case time.

Version	Best value comparisons	Worst value comparisons
Insertion by shifting	$n - 1$	$n(n-1)/2$
Bubble with fixed passes	$n(n-1)/2$	$n(n-1)/2$
Selection	$n(n-1)/2$	$n(n-1)/2$

5.2 Duplicate detection and string matching

For arbitrary comparable values, sort and scan adjacent entries to find a duplicate in $O(n \log n)$ time. Equal values are consecutive in sorted order, so a duplicate exists exactly when some adjacent pair is equal.

For a pattern of length m in a text of length n, where $1 \le m \le n$, naive string matching tries every start $1, \ldots, n - m + 1$. At each start, compare corresponding characters until a mismatch or a complete match. This tests every possible occurrence, including overlaps. Pattern `ABA` occurs at positions $1, 3, 5$ in `ABABABA`.

Worst case: $(n - m + 1)$ starts $\times$ m comparisons $= \Theta((n - m + 1)m)$, hence $O(nm)$.

- Reporting all matches: an all-`A` text and pattern attain the bound.
- Stopping at the first match: an all-`A` text and a pattern ending in `B` force every start to fail at its last character.

For $m = n$, there is one start, giving $\Theta(n)$ time.

Other counting examples.

- Uniform target position: linear search averages $(1 + \cdots + n)/n = (n + 1)/2$ comparisons.
- Boolean matrix multiplication: each entry uses n ANDs and n ORs with an accumulator.
- Matrix chain with dimensions 30×20, 20×40, 40×10: $(AB)C$ uses 36,000 multiplications; $A(BC)$ uses 14,000.

6 Optional: Selection by rank

The median-of-medians algorithm finds any rank in linear worst-case time [2].

6.1 Finding the median without sorting

Selection by rank. Select(A, k) returns the kth smallest value, counting repetitions. We use rank $\lceil n/2 \rceil$ for the median (the lower middle value when n is even). Averaging both middle values instead requires two selection calls.

For $A = [8, 3, 12, 5, 9, 2, 7]$, the median is 7: sorting gives $[2, 3, 5, 7, 8, 9, 12]$. Sorting first gives an $O(n \log n)$ algorithm. We can do better by finding just the desired rank.

Choose a pivot value p from the input, and scan once to construct three lists: L holds values below p, E holds values equal to p, and G holds values above p. Preserve multiplicities; these are lists, not mathematical sets. Then

$$\text{Select}(A, k) = \begin{cases} \text{Select}(L, k), & k \le |L|, \\ p, & |L| < k \le |L| + |E|, \\ \text{Select}(G, k - |L| - |E|), & k > |L| + |E|. \end{cases}$$

In sorted order, L comes before E, then G. Subtract the sizes of skipped lists to obtain the new rank. Keeping all equal values in E handles duplicates.

With pivot 5 in the seven-element example, $L = [3, 2]$, $E = [5]$, and $G = [8, 12, 9, 7]$. To find rank 4, continue with rank $4 - 2 - 1 = 1$ in G, whose minimum is 7. With pivot 8 in G, the smaller list is $[7]$, so the next call returns 7. This trace illustrates the rank update; it does not prescribe the pivot rule.

6.2 Median of medians: a linear-time pivot rule

An arbitrary pivot does not guarantee linear time: repeated extreme pivots can leave almost all values and cause quadratic work. The median-of-medians rule guarantees that a fixed fraction of candidates can be discarded.

1. Split the input into full groups of five and at most four leftover values. Sort each full group and collect its middle value in a list M.
2. Recursively select the lower median of M, obtaining pivot p.
3. Partition *all* original values, including leftovers, into L, E, G and continue only in the list containing rank k.

Sort directly when $n \le 5$. Here is the complete procedure, with three-way partitioning to handle ties:

procedure Select(A, k) $\quad (1 \le k \le |A|)$
if $|A| \le 5$: sort A; **return** $A[k]$
$M \leftarrow$ middle values of sorted full groups of five
$p \leftarrow$ Select$(M, \lceil |M|/2 \rceil)$
partition all of A into lists $L < p$, $E = p$, $G > p$
if $k \le |L|$: **return** Select(L, k)
if $k \le |L| + |E|$: **return** p
return Select$(G, k - |L| - |E|)$

Correctness. Both recursive inputs are smaller, since E contains the pivot. Recursion terminates, and the rank formula preserves the required answer.

A concrete grouping. Each column below is one sorted group, arranged by its middle value for illustration. The algorithm does not sort the list of group medians; it selects its median recursively.

1	3	6	8	7
2	4	10	13	11
5	**9**	**12**	**15**	**16**
14	18	19	17	20
25	24	23	21	22

The pivot is 12, the median of $5, 9, 12, 15, 16$. The first three groups give nine values ≤ 12; the last three give nine values ≥ 12. The actual median is 13: the pivot only needs to guarantee progress.

Discard guarantee. Put $m = \lfloor n/5 \rfloor$. At least $\lceil m/2 \rceil$ group medians are at most p, and at least that many are at least p. Each such full group contributes three values on the corresponding weak side. Therefore both strict lists satisfy

$$|L|, |G| \le n - 3\lceil m/2 \rceil \le 7n/10 + 3/2.$$

The last inequality uses $m \ge n/5 - 1$. It remains valid with duplicates: the certified values are counted with multiplicity, and an entire equal block is discarded when we continue on either strict side. When n is divisible by five, the cleaner bound is $|L|, |G| \le 7n/10$.

Why the total work is linear. Grouping and partitioning cost $O(n)$. The recursive input sizes satisfy

$$\underbrace{n/5}_{\text{find pivot}} + \underbrace{7n/10 + 3/2}_{\text{find answer}} \le 0.95n \qquad (n > 40).$$

Stop expanding the call tree at size 40; each such subproblem costs constant time. Across levels, the total active input size shrinks geometrically:

$$n, \quad 0.95n, \quad 0.95^2 n, \quad \ldots.$$

Charging each node's work, including terminal subproblems, by a constant c times its size gives

$$T(n) \le cn \sum_{j \ge 0} 0.95^j = 20cn = O(n).$$

The top-level scan takes $\Omega(n)$, so the worst-case time is $\Theta(n)$.

Practice. For $L = [2, 1]$, $E = [5, 5, 5]$, and $G = [9, 8]$, which call answers rank 4? Return 5, because ranks $3, 4, 5$ are in E. For rank 6, select rank $6 - 2 - 3 = 1$ in G, returning 8.

7 Exercises

1. Trace selection sort on $[4, 2, 5, 1, 3]$. Count the comparisons and swaps.
2. Merge $[1, 4, 7]$ and $[2, 3, 8]$. How many comparisons are needed?
3. Trace binary search for 6 in $[2, 3, 5, 7, 8, 9, 12]$.
4. Compare the two query strategies when $q = 1$, $q = \lceil \log_2 n \rceil$, and $q = n$.
5. Why does a direct-address table depend on U, rather than only on n?
6. Trace insertion sort on $[4, 1, 3, 2]$ and count value comparisons.
7. Count all-match string comparisons for text `AAAAAA` and pattern `AAA`. What changes if we stop at the first match?
8. In selection, let $L = [2, 1]$, $E = [5, 5, 5]$, $G = [9, 8]$. Find ranks 4 and 6.

Answers

1. The passes give $[1, 2, 5, 4, 3]$, the same list, $[1, 2, 3, 4, 5]$, and the same list. There are $4+3+2+1 = 10$ comparisons and two actual swaps.
2. The merged list is $[1, 2, 3, 4, 7, 8]$. Five comparisons select the first five values; append 8.
3. Probe positions $4, 2, 3$, holding $7, 3, 5$. The remaining interval is empty, so return -1.
4. Respectively: $\Theta(n)$ versus $\Theta(n \log n)$; both $\Theta(n \log n)$; and $\Theta(n^2)$ versus $\Theta(n \log n)$.
5. It allocates one flag for each possible value, including values absent from the input. Its initialization and storage are proportional to U.
6. The passes are $[1, 4, 3, 2]$, $[1, 3, 4, 2]$, $[1, 2, 3, 4]$. They use $1, 2, 3$ value comparisons, totaling 6.
7. Reporting all four matches takes $4 \times 3 = 12$ comparisons. Stopping at the first takes 3. For a worst case with early return, pattern `AAB` forces twelve comparisons and no match.
8. Ranks $3, 4, 5$ belong to E, so rank 4 returns 5. Rank 6 becomes rank 1 in G, returning 8.

Interactive algorithm lab: https://wuwuz.github.io/COMP2711/algorithm-lab/

References

[1] Kenneth H. Rosen. *Discrete Mathematics and Its Applications.* McGraw-Hill Education, 8th edition, 2019.

[2] Carnegie Mellon University. 15-451 lecture notes: Introduction and linear-time selection. https://www.cs.cmu.edu/~15451-f25/notes.pdf, 2025.